

地端 Agent

Learn From Claude Code

用本地模型實作 AI Agent 教程

好的學習素材

GitHub: <https://github.com/shareAI-lab/learn-claude-code>

- 這是學習如何打造 AI Agent 的優質開源教程
- 原版使用 **Anthropic Claude** API
- 架構清晰、循序漸進、程式碼簡潔

本教程將原版改寫成使用**地端模型**，
讓你不需要 API key，在本機就能跑起 Agent。

我們的環境

地端模型：google/gemma-4-26B-A4B-it-AWQ-4bit
推論引擎：vllm (OpenAI-compatible API)
API 位址：http://localhost:8999/v1

原版 (Anthropic SDK) → 改寫 (OpenAI SDK)

原版	地端版
<code>from anthropic import Anthropic</code>	<code>from openai import OpenAI</code>
<code>response.stop_reason == "tool_use"</code>	<code>finish_reason == "tool_calls"</code>
<code>input_schema: {...}</code>	<code>parameters: {...}</code>
<code>{"type": "tool_result", ...}</code>	<code>{"role": "tool", ...}</code>

Agent = LLM + Harness



LLM 負責思考，Harness 負責行動與記憶

Harness 可以包含哪些東西

元件	功能
Agent Loop	不斷呼叫 LLM 直到任務完成
Tool Use	給 LLM 使用工具的能力（bash、讀寫檔案）
Todo List	讓 LLM 追蹤自己的進度
Subagent	用子 agent 隔離複雜子任務的 context
Skill Loading	按需載入專業知識，節省 token
Context Compact	壓縮舊對話，讓 agent 能無限執行

本教程選這六個，從最簡單開始，逐步疊加

S01 — Agent Loop

為何需要？

LLM 單次呼叫只能回應一次。

但現實任務需要**多步驟、多輪工具呼叫**才能完成。

使用者：幫我找出目前目錄下最大的檔案

LLM 第一輪：呼叫 `bash("ls -lh")`

LLM 第二輪：看到結果，呼叫 `bash("du -sh *")`

LLM 第三輪：分析完畢，回答使用者

沒有 loop，LLM 就只能猜答案，不能真正執行任務。

S01 — Agent Loop 關鍵程式碼

```
def agent_loop(messages: list):
    while True:
        response = client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "system", "content": SYSTEM}] + messages,
            tools=TOOLS,
            max_tokens=8000,
        )
        msg = response.choices[0].message
        finish_reason = response.choices[0].finish_reason

        messages.append({"role": "assistant", "content": msg.content, ...})

        if finish_reason != "tool_calls":
            return # ← LLM 決定停止

        for tc in msg.tool_calls:
            output = run_bash(tc.function.arguments)
            messages.append({ # ← 結果餵回去
                "role": "tool",
                "tool_call_id": tc.id,
                "content": output,
            })
        # 繼續下一輪 ↑
```

S02 — Tool Use

為何需要？

只有 `bash` 的 agent 能做的事有限。

真實任務需要更多工具：讀檔、寫檔、編輯。

核心設計：dispatch map

```
工具名稱  →  對應函式
bash       →  run_bash()
read_file  →  run_read()
write_file →  run_write()
edit_file  →  run_edit()
```

Loop 完全不需要改，只需增加工具。

這就是 Harness 的擴展性。

S02 — Tool Use 關鍵程式碼

```
TOOL_HANDLERS = {
    "bash": lambda **kw: run_bash(kw["command"]),
    "read_file": lambda **kw: run_read(kw["path"], kw.get("limit")),
    "write_file": lambda **kw: run_write(kw["path"], kw["content"]),
    "edit_file": lambda **kw: run_edit(kw["path"], kw["old_text"],
                                       kw["new_text"]),
}

# Loop 裡只需要一行 dispatch:
for tc in msg.tool_calls:
    args = json.loads(tc.function.arguments)
    handler = TOOL_HANDLERS.get(tc.function.name)
    output = handler(**args) if handler else f"Unknown tool"
    messages.append({
        "role": "tool",
        "tool_call_id": tc.id,
        "content": str(output),
    })
```

S03 — Todo List

為何需要？

LLM 執行長任務時容易忘記做到哪裡。

沒有追蹤機制，它可能：

- 重複做同一件事
- 跳過某些步驟
- 不知道什麼時候算完成

解法：讓 LLM 自己維護一份 todo 清單

```
[>] #1: 讀 s01_agent_loop.py    ← 進行中
[x] #2: 讀 s02_tool_use.py      ← 完成
[ ] #3: 寫總整理報告           ← 待辦
```

S03 — Todo List 關鍵程式碼

```
class TodoManager:
    def update(self, items: list) -> str:
        # 驗證：只允許一個 in_progress
        in_progress = [i for i in items if i["status"] == "in_progress"]
        if len(in_progress) > 1:
            raise ValueError("Only one task can be in_progress")
        self.items = items
        return self.render()          # ← 回傳清單讓 LLM 看到

# 每 3 輪沒更新 todo, 自動提醒:
rounds_since_todo = 0 if used_todo else rounds_since_todo + 1
if rounds_since_todo >= 3:
    messages.append({
        "role": "user",
        "content": "<reminder>Update your todos.</reminder>"
    })
```

S04 — Subagent

為何需要？

複雜任務拆成子任務，但主 agent 的 context 會越來越大。
子任務的執行細節污染主 agent 的思維。

解法：子任務用全新的 context 執行

```
主 agent (context 保持乾淨)
├── task("分析 s01.py") → 子 agent (fresh context)
│                           執行、回傳摘要
│                           context 丟棄
└── task("分析 s02.py") → 子 agent (fresh context)
│                           執行、回傳摘要
│                           context 丟棄
```

S04 — Subagent 關鍵程式碼

```
def run_subagent(prompt: str) -> str:
    sub_messages = [{"role": "user", "content": prompt}] # fresh!

    for _ in range(30): # 安全上限
        response = client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "system", "content": SUBAGENT_SYSTEM}]
                + sub_messages,
            tools=CHILD_TOOLS, # 沒有 task 工具 (不能遞迴)
            max_tokens=8000,
        )
        ...
        if finish_reason != "tool_calls":
            break

    # 只回傳最後的文字摘要給主 agent
    return response.choices[0].message.content or "(no summary)"
    # sub_messages 在這裡被丟棄 ↑
```

S05 — Skill Loading

為何需要？

把所有知識塞進 system prompt → **token 爆炸**。

Agent 可能需要很多領域知識（git、pdf、docker...），
但大多數時候只用到其中一兩個。

兩層注入策略：

Layer 1（便宜）：只在 system prompt 放 skill 名稱 + 一行描述
~100 tokens/skill

Layer 2（按需）：LLM 主動 `load_skill("git")` 時，
才把完整指南放進 `tool_result`
~1000 tokens，只在需要時花

S05 — Skill Loading 關鍵程式碼

```
# skills/git/SKILL.md (YAML frontmatter + body)
# ---
# name: git
# description: Git workflow – commits, branches, diffs
# ---
# ## Git Workflow ...

class SkillLoader:
    def get_descriptions(self) -> str:
        # Layer 1: 只給名稱和描述 → system prompt
        return " - git: Git workflow – commits, branches..."

    def get_content(self, name: str) -> str:
        # Layer 2: 完整內容 → tool_result (按需)
        return f"<skill name='{name}'>\n{body}\n</skill>"

# System prompt 只放 Layer 1:
SYSTEM = f"""...
Skills available:
{SKILL_LOADER.get_descriptions()}"""
```

S06 — Context Compact

為何需要？

LLM 的 context window 有上限。
長任務執行到一半，對話累積太多 → 報錯或品質下降。

三層壓縮策略：

層	觸發	動作
micro_compact	每輪自動	舊 tool result 換成 [Previous: used bash]
auto_compact	tokens 超過門檻	LLM 摘要整段對話，替換 messages
compact tool	LLM 主動呼叫	同 auto，但 LLM 可指定保留重點

S06 — Context Compact 關鍵程式碼

```
def agent_loop(messages):
    while True:
        # Layer 1: 靜默替換舊 tool result
        micro_compact(messages)

        # Layer 2: 超過門檻自動壓縮
        tokens = estimate_tokens(messages)
        print(f"[context: ~{tokens} tokens]")
        if tokens > THRESHOLD:
            messages[:] = auto_compact(messages)
            # TodoManager 在 Python memory, 不受影響 ↑

        response = client.chat.completions.create(...)
        ...

def auto_compact(messages):
    # 1. 存完整 transcript 到磁碟
    # 2. 叫 LLM 做摘要 (保留 todo 狀態)
    # 3. 用一條 user 訊息取代整個 messages[]
    return [{"role": "user", "content": summary + todo_state}]
```

Todo 狀態為何能存活壓縮？

```
messages[]          TodoManager.items
(壓縮後只剩摘要)    (Python 物件，永遠活著)

[摘要一條]   vs   [x] #1: 讀 s01
                  [x] #2: 讀 s02
                  [ ] #3: 讀 s03 ← 醒來繼續這裡
                  [ ] #4: 寫報告
```

壓縮後 model 第一件事：呼叫 `todo()`，
看到還有 pending 項目，繼續執行。

關鍵設計原則：重要狀態不要放在 `messages` 裡。

教程總結

主題	核心概念
S01 Agent Loop	<code>while finish_reason == "tool_calls"</code>
S02 Tool Use	dispatch map, loop 不變只加工具
S03 Todo List	LLM 自己追蹤進度 + nag reminder
S04 Subagent	fresh context, 只回傳摘要
S05 Skill Loading	兩層注入, 按需載入
S06 Context Compact	三層壓縮, todo 在 messages 外存活

原始碼： `github.com/shareAI-lab/learn-claude-code`

地端版： 本教程 repo (gemma-4 + vllm)